

```
New permutation: ADCB
New permutation: DACB
New permutation: DCAB
New permutation: DCBA
When chars = ABCD results are ['ABCD', 'BACD', 'BCAD', 'BCDA', 'ACBD', 'CABD', 'CBAD', 'CBDA', 'ACDB', 'CADB', 'CDAB', 'CDBA', 'ABDC', 'BADC', 'BDAC', 'BDCA', 'ADBC', 'DABC', 'DBAC', 'DBCA', 'ADCB', 'DACB', 'DCAB', 'DCBA']
Results: ABCD, BACD, BCAD, BCDA, ACBD, CABD, CBAD, CBDA, ACDB, CADB, CDAB, CDBA, ABDC, BADC, BDAC, BDCA, ADBC, DABC, DBAC, DBCA, ADCB, DACB, DCAB, DCBA
```

---

When `getPerms()` is called, it first checks for the base case ❶. If the `chars` string is only one character long, it can have only one permutation: the `chars` string itself. The function returns this string in an array.

Otherwise, in the recursive case, the function splits the `chars` argument's first character into the `head` variable and the rest into the `tail` variable ❷. The function makes a recursive call to `getPerms()` to get all the permutations of the string in `tail`. A first `for` loop ❸ iterates over each of these permutations, and a second `for` loop ❹ creates a new permutation by placing the `head` character in every possible place in the string.

For example, if `getPerms()` is called with `ABCD` for the `chars` argument, `head` is `A` and `tail` is `BCD`. The `getPerms('BCD')` call returns an array of the tail permutations, `['BCD', 'CBD', 'CDB', 'BDC', 'DBC', 'DCB']`. The first `for` loop starts with the `BCD` permutation, and the second `for` loop places the `A` string in `head` in each possible place, producing `ABCD, BACD, BCAD, BCDA`. This is repeated with the remaining tail permutations, and the entire list is then returned by the `getPerms()` function.